



LEGO: Supporting LLM-enhanced Games with One Gaming GPU

Han Zhao^{*¶}, Weihao Cui^{*¶}, Zeshen Zhang[◇], Wenhao Zhang^{*}, Jiangtong Li[◇], Quan Chen^{*},
Pu Pang^{*}, Zijun Li^{*}, Zhenhua Han[†], Yuqing Yang[‡], Minyi Guo^{*}

^{*}Shanghai Jiao Tong University [†]National University of Singapore [◇]Tongji University

[†]Shanghai Qiji Zhifeng Co., Ltd. [‡]Microsoft Research [¶]Equal Contribution

Abstract—Artificial intelligence (AI) has been increasingly applied to gaming, with large language models (LLMs) playing a key role in character control. However, efficiently co-locating game rendering and LLM inference on one GPU presents challenges due to resource constraints, diverse latency requirements, and fine-grained task scheduling. We propose LEGO, an algorithm-system co-design that enables the efficient co-location of LLM inference and game rendering tasks. Algorithm-wise, LEGO features a resource-oriented layer-skipping adaptor, which distills knowledge from skipped layers to reduce computational demand while maintaining inference accuracy. System-wise, LEGO proposes a headroom-maximizing LLM scheduler, which dynamically partitions inference tasks to utilize available rendering headroom. Evaluations on an Nvidia RTX 4090 show that LEGO meets latency targets in all scenarios, improves rendering headroom utilization by up to 28.6%, and reduces LLM inference accuracy loss by up to 86.3% compared to current layer-skipping approaches.

I. INTRODUCTION

Using artificial intelligence (AI) algorithms to enhance games has long been a prominent area of research [7], [36]. The recent emergence of large language models (LLMs) has opened up new possibilities for AI applications in gaming [48], [53], [63]. For example, the Open Generative AI community uses *Street Fighter III* [3] to evaluate the action skills of LLMs. Similarly, researchers at Alibaba have explored the use of LLMs to play the action game *Black Myth: Wukong* (*BlackMyth*) [11]. In these scenarios, the LLM receives environment information and character status as prompts and generates combat actions.

Existing research typically employs separate hardware to support game rendering and LLM inference tasks. Figure 1 illustrates the separate deployment of a popular game, *BlackMyth*, alongside an LLM inference task using Llama3-8B [16]. As shown, both game rendering and LLM inference tasks exhibit periodic execution patterns. The default configuration of *BlackMyth* is 60 frames per second (FPS), meaning a rendering task is generated every 16.6 ms and should be completed within the same 16.6 ms deadline. Meanwhile, LLM inference is designed to simulate player actions at varying skill levels, characterized by actions per minute (APM) [1], [2], [7]. An average player has an APM of approximately 100, an excellent player around 200, and a professional player about 300. Under the 100 APM scenario, an LLM inference task is generated every 600 ms, with a latency target of 600 ms.

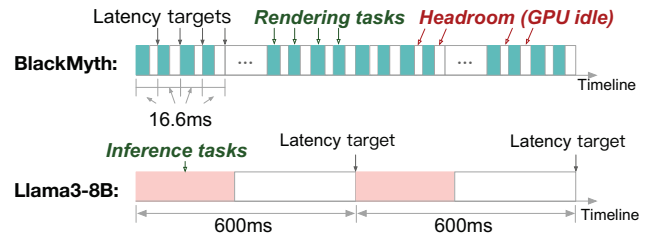


Fig. 1: The separate deployment of a game *BlackMyth* and an LLM inference task using *Llama3-8B*.

However, existing deployment strategies are not feasible on the client side, as most users have only one GPU on their personal machines. In this case, a natural idea is to leverage cloud-based LLM services. Unfortunately, the end-to-end network overhead of cloud LLM services typically ranges from 20ms to 110ms [6], [10], [32], [54]. This is unacceptable for gaming scenarios, where 200 APM and 300 APM scenarios require SLOs (Service-Level Objective) of 300 ms and 200 ms, respectively. More critically, relying on cloud-based LLM services increases the overall cost of the game and undermines its market competitiveness.

Meanwhile, we observe considerable underutilization of GPU resources when the rendering task runs alone. Experimental results on an Nvidia RTX 4090 show that *BlackMyth* with high visual settings utilizes only 60.8% of the GPU time. This underutilization suggests a promising opportunity to co-locate game rendering and LLM inference on the same gaming GPU. However, effectively leveraging this opportunity is non-trivial, as the available compute headroom is insufficient, dynamic, and fragmented.

Although 39.2% of the GPU time slice appears idle in *BlackMyth*, running Llama3-8B [16] in a 100 APM scenario requires 41.9% of the GPU time—exceeding the available capacity. The resource gap only widens under 200 APM and 300 APM scenarios. Moreover, LLM inference relies on compute headroom from multiple rendering tasks for computation. Direct co-location leads to disordered contention, causing latency violations for rendering tasks. Therefore, effective co-location demands fine-grained task scheduling, which is challenging.

Since the highest priority in gaming is to ensure the player's visual experience, we should reduce the computational demands of LLM inference under limited resources. Faced

with this demand, layer-skipping [52], [58] and quantization [24], [41], [46] techniques are two potential solutions. However, current GPUs only support limited formats, which means several fixed resource usage levels for LLM inference task. This lack of flexibility makes quantization poorly suited for dynamic resource conditions in task co-location within gaming. Therefore, in this work, we focus on layer-skipping techniques as a more adaptable solution.

Existing layer-skipping methods [52], [58] rely on runtime discrimination mechanisms to make per-token layer-skipping decisions. These methods typically optimize for the average number of skipped layers across all tokens, rather than enforcing guarantees for individual tokens. As a result, they easily lead to latency violations under strict SLO constraints. Meantime, adapting these methods to enforce strict SLO guarantee requires skipping layers in advance. This results in significant accuracy degradation, as they may skip layers that are deemed important by their own mechanisms.

To this end, we propose LEGO, an algorithm–system co-design approach that maintains inference accuracy while satisfying the SLOs of both LLM inference and rendering tasks. Algorithm-wise, LEGO proposes a resource-oriented layer-skipping technique that mitigates accuracy degradation when layer-skipping decisions are made based solely on resource availability. System-wise, LEGO designs a headroom-maximizing scheduling strategy that enables LLM inference to fully utilize available compute resources, thereby further reducing the need for layer skipping.

Specifically, LEGO first proposes the layer-skipping adaptor for task co-location in gaming. Inspired by knowledge distillation methods, the adaptor distills information from the skipped layers. For each possible layer-skipping configuration, LEGO identifies the less important layers and trains an adaptor (a feed-forward network) to distill knowledge from them. This design mitigates the loss of critical information caused by SLO-driven layer skipping, thereby preserving the quality of inference. For runtime scheduling, LEGO then designs an LLM scheduler based on two observations: 1) rendering headroom exists not only in the gaps between consecutive rendering tasks but also within individual rendering tasks; 2) the overall headroom across these tasks can be effectively estimated, while accurately predicting per-task compute headroom of multiple consecutive rendering tasks is challenging.

At runtime, the LLM scheduler employs a linear regression (LR) model for rendering headroom prediction. The model takes the overall headroom from the previous three inference windows to predict the headroom of the next one. Based on the prediction, the scheduler determines the appropriate layer-skipping strategy for the upcoming LLM inference task. After determining the layer skipping, the scheduler splits each LLM inference task into smaller subtasks to make use of fragmented GPU headroom. For intra-rendering headroom, it monitors the start and end of rendering subtasks. When no rendering subtasks are running, the scheduler dispatches fine-grained LLM subtasks to utilize this headroom. Once a rendering task completes, it switches to coarse-grained LLM subtasks

TABLE I: Representative games that using LLM at runtime.

Game	Year	Runtime LLM Usage
AI Roguelite	2023	Live-generate text and mechanics decisions
Vaudeville	2023	Dialogues generated in real time
AI Game Master	2025	Procedural quests/characters
inZOI	2025	LLM-driven NPCs, player-prompt generate
PUBG Ally	2025	Co-playable LLM agent companion
Astrobuilder	2025	NPC behavior/strategic guidance
Mecha BREAK	2025	Conversational NPCs via NVIDIA ACE
AI2U	2025	NPC dialogue & voice generated via LLM
EmemeTown	2025	NPC conversations generated in real time
Life of an NPC	2025	LLM-directed town

to better use the larger headroom between rendering tasks.

We evaluate LEGO using several popular games and LLM models on a mainstream gaming GPU, the Nvidia RTX 4090. Experimental results show that LEGO consistently meets the latency targets for both rendering and LLM inference across all APM scenarios. In addition, LEGO improves rendering headroom utilization by up to 28.6% and reduces accuracy loss by up to 86.3% compared to existing layer-skipping methods. Our key contributions are as follows:

- We present a practical solution for integrating LLMs into games without incurring the latency and cost penalties. This would have a positive impact on the gaming industry.
- We design a resource-oriented layer-skipping adaptor to distill knowledge from the skipped layers, which could reduce the latency drop of LLM.
- We propose a headroom-maximizing LLM scheduler to enable LLM inference to utilize all available rendering headroom. This helps make the optimal layer-skipping strategy.

II. BACKGROUND AND MOTIVATION

A. Trends in Using LLMs in Games

As of 2025, approximately 7% of all games on Steam (about 7,800 out of 110,000) incorporate AI technologies [27]. The proportion of newly released games adopting LLMs has grown by over 700% compared to 2024 [51]. Among these, 16 games (4 released before 2024 and 12 in 2025) explicitly report using LLMs at runtime [26], [39], [44], [45], [55], [60]. Table I shows the specific applications of representative games.

More importantly, NVIDIA has introduced the LLM-powered game companion ACE in 2023, which has attracted significant attention and adoption from game studios [43]. Meanwhile, academic interest has surged, with 57, 141, and 159 papers published in 2023, 2024, and 2025, respectively [21], [22], [28], [40], [57]. This surge in both industrial adoption and academic output underscores the increasing significance and momentum of this emerging field.

B. LLM workflow in Game

Taking the combat scenario as an example, LLMs simulate player behavior to act as characters or enemies with varying difficulty levels. Figure 2 illustrates the workflow of LLMs in such scenarios. As shown, human players continuously perform in-game actions at a certain frequency. In response, LLMs must generate actions at a comparable rate [1], [2], [7].

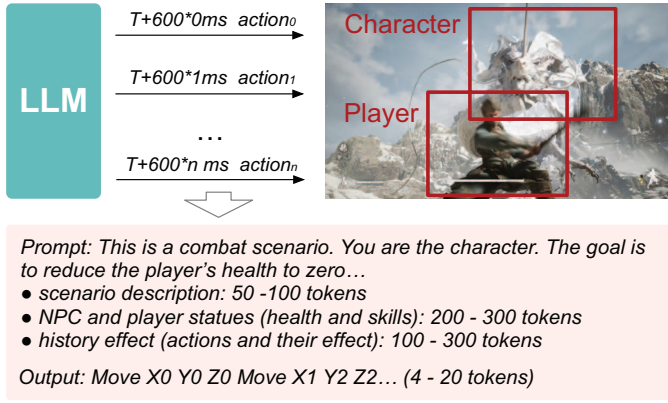


Fig. 2: The LLM workflow in combat scenarios.

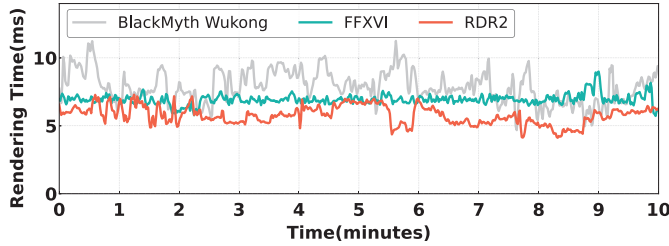


Fig. 3: The execution times of rendering tasks for three games.

The frequency of human player actions is commonly measured using the metric *Actions Per Minute (APM)*. Average players typically have an APM of 50–100, excellent players range between 150–200, and professional players achieve 250–300. To match the operational frequency of human players, the LLM’s action generation rate should adapt accordingly. In this paper, we select three representative scenarios corresponding to APM levels of 100, 200, and 300, representing average, advanced, and professional gameplay.

In the 100 APM combat scenario, the LLM should generate one action every 600 milliseconds. The lower part of Figure 2 illustrates a possible input-output pair for a single action. As shown in the figure, the LLM input consists of three aspects: (1) the current game scene’s state information, (2) the current state information of the character and player, and (3) historical action information. The LLM output contains 4–20 tokens, encoding information for one to five skills. Each skill includes its name and emission direction. Five skills represent a combo, referring to consecutive skill executions within a short time. When the LLM generates the action, the game engine executes the action in the game scene.

Following prior works [23], [61], key game states (e.g., player positions, skill states) are directly accessible from the game engine. Since the input length and output length fall within a range, we select 512 as the representative input length and 16 as the output length throughout the paper.

C. The Co-location of Game and LLM

1) *Inefficiency of Existing Deployment*: While existing research typically employs separate hardware for game rendering and LLM inference tasks, this approach is not feasible on the

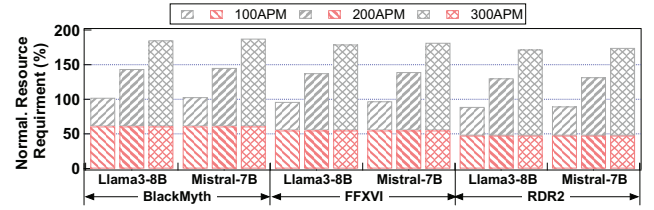


Fig. 4: The total resource requirements of Game-LLM co-location pairs across three APM scenarios.

client side, as most users have access to only a single GPU on their personal machines.

In this case, there are two possible options for the client: using cloud-based LLM services or co-locating both tasks on the same gaming GPU. However, for Azure OpenAI, network latency ranges from about 20–110 ms within nearby regions and can reach up to 300 ms across continents [54]. Meanwhile, using the OpenAI API adds around 300–700 ms of latency compared to running the model locally [6]. The network latency is unacceptable for gaming scenarios, where 200 APM and 300 APM workloads demand SLO targets of 300 ms and 200 ms, respectively [10], [32]. Moreover, LLM services rely on large batch sizes to improve resource utilization, which further increases end-to-end latency. More critically, the reliance on LLM services significantly increases the overall cost of the game and reduces its market competitiveness.

Alternatively, we examine the feasibility of local task co-location by profiling GPU utilization when game rendering runs alone. We select *Black Myth: Wukong (BlackMyth)*, *Final Fantasy XVI (FFXVI)*, *Red Dead Redemption 2 (RDR2)* as game benchmarks, and collect 30 minutes of rendering data for each. All games are configured with high visual settings (4K resolution) and run at 60 frames per second (FPS). Specifically, we measure the compute headroom between the end of one rendering task and the start of the next.

Figure 3 presents the rendering times for the three games. As shown, all rendering tasks exhibit significant compute headroom, referred to as *rendering headroom*. Specifically, the maximum rendering times for the three games are 10.1 ms, 9.1 ms, and 7.9 ms, respectively, while the SLO for rendering tasks is 16.6 ms. Over the long term, these games require GPU time slice reservations of 60.8%, 54.8%, and 47.6% to ensure all rendering tasks meet their latency targets. This suggests that co-locating game rendering and LLM inference on the same gaming GPU is a promising solution.

2) *The Challenges of Game-LLM Co-location*: Although task co-location is promising, it faces two key challenges.

First, directly co-locating rendering and LLM inference tasks exceeds GPU compute capacity. Figure 4 presents the total resource demand across three games, two LLM models (Llama3-8B [16] and Mistral-7B [4]), and three APM scenarios. In the figure, red bars represent rendering task resource consumption, while gray bars indicate LLM inference task demand. The stacked bars show the total resource demand at task co-location. As shown, 14 out of 18 scenarios exceed the

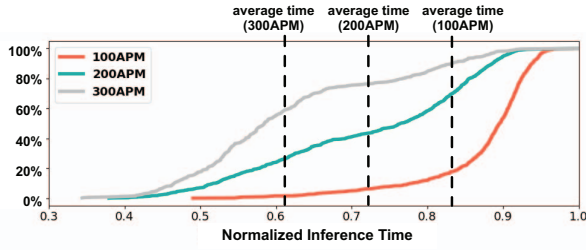


Fig. 5: The CDF of LLM inference task execution time under LITE using different threshold.

compute limit, meaning that existing GPUs lack the capacity to handle both tasks simultaneously.

Second, co-locating rendering and LLM inference tasks requires fine-grained task scheduling. Figure 1 illustrates that a single LLM inference task spans multiple rendering task periods. Direct co-location leads to severe latency violations in rendering tasks. To ensure both tasks meet their latency targets, LLM inference must utilize the fragmented headroom within rendering tasks. Moreover, Figure 3 shows that rendering task execution time fluctuates significantly, further increasing scheduling complexity.

D. Possible solutions

Since ensuring the player’s visual experience is the highest priority in gaming scenarios, we should reduce the computational demands of LLM inference tasks. To address this, three potential solutions exist: using smaller models, applying quantization techniques, and employing layer-skipping methods. However, the first two approaches lack the flexibility to adapt to dynamic resource conditions in task co-location scenarios. Therefore, in this paper, we focus on layer-skipping methods as a more adaptable solution.

1) *Smaller Model*: We collect the overall computational demand when co-locating tasks with two corresponding small-size models. Results show that, while small models can meet resource demands in the 100 APM scenario, they fail to support the 200 APM and 300 APM scenarios. Furthermore, even in the 100 APM scenario where small models can be deployed, they suffer an average inference accuracy drop of 20.41% on the MMLU, ARC-C, and SQuAD-2.0 datasets.¹

2) *Layer-skipping Methods*: Existing layer-skipping methods rely on runtime discrimination mechanisms to make per-token layer-skipping decisions. In this work, we adopt LITE [58] as the baseline method. LITE designs a finetune method for the LLM model and defines a predefined confidence threshold for each layer. At runtime, if a token’s confidence score exceeds the threshold at a given layer, LITE skips the remaining layers and outputs the token. As a result, different tokens may exit the model at different depths.

We measure the inference time of Llama3-8B [16] under LITE. Specifically, we configure LITE with corresponding

¹Currently, there is a lack of mature, standardized datasets specifically tailored for LLM-based gaming. Therefore, following prior works [8], [14], [57], we use datasets that are closely aligned with gaming-related tasks, such as role-playing and reasoning.

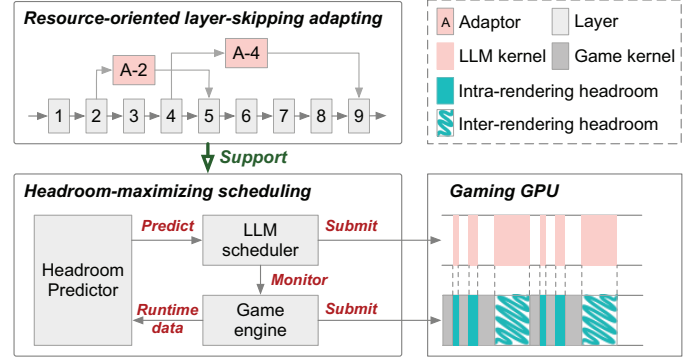


Fig. 6: The design overview of LEGO.

thresholds for different APM scenarios. This ensures the average computation time of LLM inference aligns with the latency target. Figure 5 presents the LLM inference time on the SQuAD-2.0 dataset, with all times normalized to the latency target. As shown, 47.1% of LLM inference tasks exceed the predefined latency target, leading to latency violations.

To enable LITE to meet SLO guarantees, a naive approach is to force early layer skipping when there is a risk of SLO violation. Building on the above experiment, we implement LITE-S, an extension of LITE that incorporates SLO constraints. Experimental results show that enforcing SLO guarantees leads to a 27.2% drop in accuracy, primarily because LITE-S skips layers that are considered important by its own confidence-based mechanism.

Therefore, current layer-skipping methods could not address the challenges of task co-location in gaming scenarios.

III. LEGO DESIGN

In this section, we present LEGO, an algorithm-system co-design that meets the requirements of task co-location in gaming scenarios. As shown in Figure 6, LEGO proposes a resource-oriented layer-skipping adaptor and a headroom-maximizing LLM scheduler. The adaptor enables LLM inference tasks to perform layer skipping based solely on compute resources while preserving inference accuracy as much as possible. The scheduler facilitates fine-grained scheduling of LLM inference tasks, maximizing the utilization of rendering headroom while ensuring the latency targets of both tasks.

Specifically, inspired by knowledge distillation, LEGO introduces a resource-oriented layer-skipping adaptor to distill knowledge from skipped layers. When defining a layer-skipping strategy for a specific resource usage, LEGO identifies less important layers and then employs an adaptor (a feed-forward network layer) to distill knowledge from the skipped layers. This self-distillation design caters to layer-skipping demands based on resource availability, while preserving inference quality.

With the support of the adaptor, the headroom-maximizing LLM scheduler determines the layer-skipping strategy and schedules LLM inference tasks to effectively utilize dynamic and fragmented rendering headroom at runtime. The scheduler

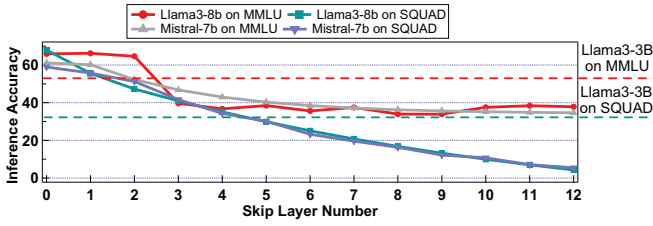


Fig. 7: The accuracy drops when skipping layers directly.

design is based on two key observations: (1) rendering headroom exists not only between rendering tasks but also within them, and (2) although predicting each compute headroom of multiple consecutive rendering tasks is difficult, the overall headroom of these tasks can be accurately estimated using a linear regression (LR) model.

At runtime, the scheduler employs the LR model to predict rendering headroom. Specifically, it uses the total headroom from the previous three inference windows to predict the available headroom for the next window. For example, in a 100 APM scenario, each inference window spans the total rendering headroom across 36 rendering tasks. Based on this prediction, the scheduler selects an appropriate layer-skipping strategy for the upcoming LLM inference.

Following the layer-skipping strategy, the scheduler splits the LLM inference task into smaller subtasks to utilize fragmented GPU headroom. For intra-rendering headroom, the scheduler monitors the start and end of rendering subtasks. When no rendering subtasks are active, it dispatches fine-grained LLM subtasks to fill these short gaps. Once a rendering task completes, it switches to coarse-grained subtasks to better utilize the larger headroom available between rendering tasks. Throughout this process, the scheduler maximizes the use of all available rendering headroom. It is worth noting that the LR model already consider the intra-rendering headroom.

Note that, LEGO is designed for commercial game companies, rather than end users. It provides a practical deployment solution: commercial companies can train their own models and adaptors, then package them together with the game. When users download the game, both the game and the LLM are deployed and ready to run locally. In addition, LEGO can be integrated into cloud gaming platforms, like Nvidia GeForce NOW [43]. In this setting, PilotFish [66] is a time-division management mechanism for cloud gaming, which can be leveraged to schedule LLM inference within the available compute headroom.

IV. LAYER-SKIPPING ADAPTOR

In this section, we first present an empirical analysis of the performance degradation caused by layer skipping based solely on resource availability. We then introduce our resource-oriented layer-skipping adaptor to mitigate the accuracy loss.

A. Performance Degradation

In Figure 7, we present the inference accuracy of two LLMs (*i.e.*, Llama3-8B and Mistral-7B), evaluated on the

MMLU and SQuAD-2.0 datasets, under varying layer-skipping configurations. For the experiment, “skipping 1 layer” denotes bypassing the final transformer layer of the LLM, thereby connecting the penultimate transformer layer directly to the output layer. Similarly, “skipping N layers” corresponds to bypassing the last N transformer layers.

For reference, the inference accuracy of the Llama3-3B and Mistral-4B models serves as the baseline, as indicated by the dashed lines. As shown in Figure 7, both Llama3-8B and Mistral-7B exhibit a pronounced drop in inference accuracy. When four layers are skipped, the accuracy falls below the baseline, indicating a substantial loss of model knowledge.

Theoretically, the inference accuracy drop implies a degradation of the latent knowledge representation in the LLM. Direct layer skipping may incur knowledge loss in two ways [17]: (1) the removal of knowledge encoded within individual transformer layers, and (2) the disruption of coherent representations between different transformer layers.

Faced with this pronounced performance degradation, we propose a two-stage mitigation strategy. First, we identify transformer layers that contribute less to the overall knowledge representation. Second, we employ a knowledge distillation technique to approximate and restore the knowledge encapsulated in the skipped layers, thus maintaining inference quality and enhancing computational efficiency.

B. Resource-oriented Layer-skipping Adapting

Given that all transformer layers share identical output tensor dimensions, previous studies [38] on LLM interpretability indicate that each transformer layer encodes a distinct knowledge representation in its output tensor. *Therefore, a high degree of similarity between the input and output tensors of a transformer layer implies that minimal new information is introduced at that layer, resulting in a reduced contribution of unique knowledge to the model.* Consequently, we begin by quantifying the similarity among the transformer layers.

In Figure 8, we illustrate the similarity heatmaps for Llama3-8B and Mistral-7B, in which the output tensors across all transformer layers are compared. Each block represents the similarity between the output tensor T_i of transformer layer L_i and the output tensor T_j of transformer layer L_j . For example, the block enclosed by a green square in the bottom-left corner corresponds to the similarity between the output tensor T_1 of layer L_1 and the output tensor T_2 of layer L_2 . Notably, the experiment for Figure 8 uses 2400 samples from the WebInstruct dataset. For each sample, we use the first 16 output tokens, consistent with our game setting.

From Figure 8, we derive three critical observations. First, the diagonal lines in the heatmap reflect the various layer-skipping configurations. For instance, the blocks along the green diagonal correspond to the Layer pairs (L_j, L_i) where $j-i=4$, meaning they collectively represent all possible 4-layer skip candidates. Secondly, both LLMs demonstrate high inter-layer similarity in the latter layers of the network, in contrast to the lower similarity observed in the initial layers, thereby suggesting that omitting several consecutive layers in

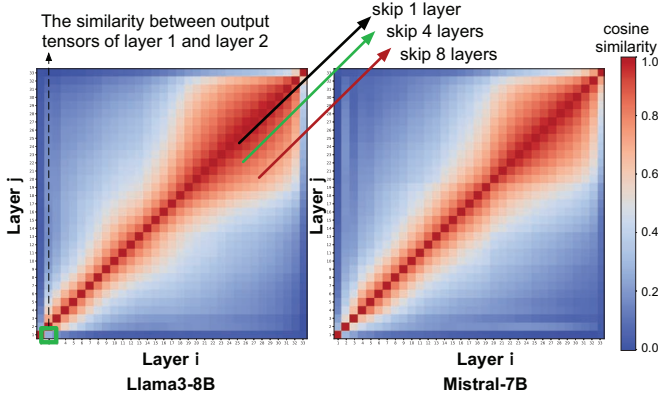


Fig. 8: The similarity heatmaps for Llama3-8B and Mistral-7B, comparing output tensors across all transformer layers.

the later layers is a feasible approach. Finally, the output of the last transformer layer exhibits low similarity to that of the penultimate layer. Considering that the final transformer layer encodes crucial knowledge for interfacing with the output layer, it should not be skipped.

Based on these observations, we propose a resource-oriented layer-skipping adapter, implemented as an FFN layer, to replace a block of consecutive transformer layers. In particular, when skipping N layers, we identify the contiguous layer range that exhibits the highest similarity along the diagonal of the similarity heatmap. For example, when skipping four layers for Llama3-8B, the layer range from L_{25} to L_{29} displays the highest similarity. Similarly, when skipping eight layers, the layer range from L_{23} to L_{31} exhibits the highest similarity.

In Figure 9, we present two examples that illustrate the adaptor. When skipping layers during inference, an additional FFN layer is used to approximate the knowledge representation originally encapsulated by the skipped transformer layers.

C. Prepare Adaptors Offline

Preparing the adaptors first requires identifying the possible layer-skipping range for a specific game, followed by training the adaptors on a designated dataset.

1) *Perceive the layer-skipping range*: Determining the layer-skipping range can be divided into three steps.

Step 1: We profile the execution times of rendering tasks over a representative gameplay period. Based on these measurements, we compute the minimum and maximum computational headroom (i.e., C_{min} and C_{max}) for the rendering tasks. Using these values, we then calculate the minimum and maximum rendering headroom (H_{min} and H_{max}) available for LLM inference under different APM scenarios.

Step 2: We measure the overall computational time $T_{overall}$ for an LLM inference action, and profile the prefill phase (T_{pi}) and the decode phase (T_{dl}) for each transformer layer.

Step 3: Using the profiling results, we compute the minimum number of layers M that must be skipped to satisfy the minimum rendering headroom, and the maximum number N under the maximum rendering headroom. Consequently, this

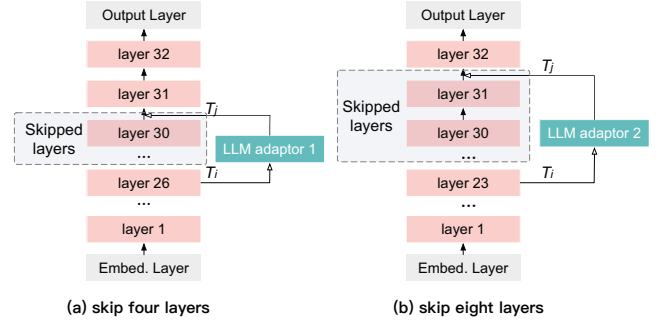


Fig. 9: The LLM adaptor examples.

approach yields $N - M + 1$ distinct layer-skipping strategies, each of which requires a separately trained adaptor.

2) *Train the adaptors on the dataset*: Generally, game companies build their LLMs by fine-tuning a mature base model, such as Llama3-8B, on their private datasets. Based on the resulting fine-tuned model, we first construct the inter-layer similarity heatmap using the same dataset. Specifically, we first compute the cosine similarity between transformer layers for each data item, and then produce the similarity heatmap by averaging these similarities across the entire dataset.

Next, for each specific layer-skipping strategy, we analyze the similarity heatmap to identify a contiguous range of layers with the highest similarity. For example, in a particular layer-skipping case—i.e., skipping from the k -th layer to the $k+n$ -th layer—we employ an adaptor (a feed-forward network, FFN) [59] to approximate the transformation performed by the skipped layers (from k -th to $k+n$ -th). The adaptor is trained by minimizing the Mean Squared Error (MSE) loss between its output and the original output of the skipped layers, which is formulated as $\mathcal{L}_{mse} = \|f_{k+n} - \text{FFN}_k^{k+n}(f_k)\|_2^2$, where f_{k+n} and f_k are the output features of the $(k+n)$ -th and k -th layers, respectively. FFN_k^{k+n} denotes the corresponding adaptor network.

The above training process updates only the weights of the adaptor, which contains a relatively small number of parameters, making training efficient. Moreover, the process can be further accelerated by reusing intermediate outputs from earlier layers across different adaptors, reducing redundant computation. For example, in the case of *BlackMyth*, up to 14 LLM adaptors are required, and the total training time is approximately 36 hours.

V. HEADROOM-MAXIMIZING SCHEDULER

In this section, we first conduct an in-depth analysis of rendering headroom. Next, we design a rendering headroom prediction model to support runtime scheduling. Finally, we propose a headroom-maximizing strategy to effectively utilize the dynamic and fragmented rendering headroom.

A. Headroom Analysis

Figure 10(a) illustrates the scheduling requirements for LLM inference tasks under task co-location. As shown, executing an LLM inference task requires leveraging the compute headroom from multiple rendering tasks within its execution

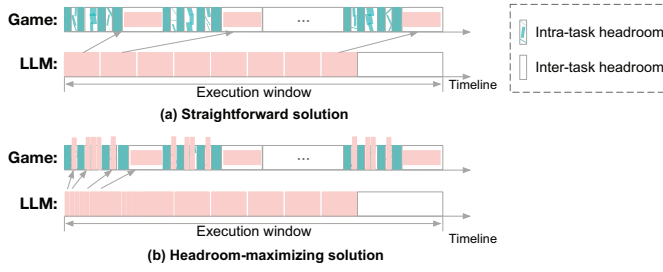


Fig. 10: The scheduling solutions for the task co-location.

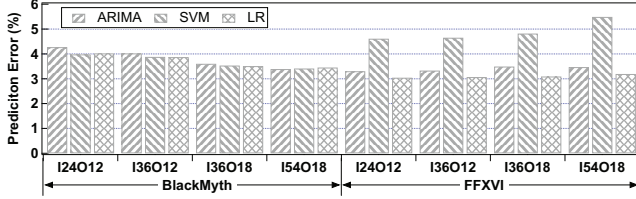


Fig. 11: The prediction accuracy of three time-series models under various configurations.

window. One straightforward method is to adopt PilotFish’s method [66] to monitor rendering task completion. Once a rendering task completes, the scheduler calculates the available compute headroom based on the current timestamp and dispatches an appropriately sized LLM inference subtask.

While this method efficiently utilizes compute headroom between rendering tasks, we observe that limited compute resources can lead to excessive layer skipping in LLM inference tasks. For instance, in the 300 APM scenario across all three games, LLM inference tasks skip over 50% of transformer layers. Even though the LLM adaptor distills knowledge from skipped layers, inference accuracy still drops by 70%.

To address this issue, we further analyze GPU utilization under task co-location. We dive into the task trace using Nvidia’s profiling tool *nsight-system* [5]. Experimental results reveal that while LLM inference tasks continuously occupy the GPU, rendering tasks do not. As shown in Figure 10(a), significant GPU idle time exists within rendering tasks themselves.

Further investigation reveals that intra-rendering headroom arises from optimizations within the game engine. Specifically, game engines optimize rendering by batching similar objects, accelerating the rendering pipeline. As a result, a rendering task consists of multiple subtasks, where some perform computations (rendering subtasks) and others handle auxiliary operations that do not use the GPU (auxiliary subtasks).

B. Headroom Prediction

Before layer-skipping strategy selection, the scheduler must predict the total rendering headroom within the next execution window. A naive approach is to use existing time-series models to predict the headroom of each rendering task within the window and then compute the total available headroom.

Figure 11 shows the prediction accuracy of three time-series models under various configurations. For example, I24-O12

TABLE II: The prediction errors using LR model with the execution window as a time unit across all scenarios.

Scenarios	100APM	200APM	300APM
BlackMyth	0.44%	0.87%	1.31%
FFXVI	0.19%	0.38%	0.55%
RDR2	0.27%	0.54%	0.81%

indicates a setup where the headroom values from 24 previous rendering tasks are used to predict the headroom of the next 12. These configurations correspond to the window sizes of different APM scenarios: 36 tasks for 100 APM, 18 for 200 APM, and 12 for 300 APM. We do not present the results of 100 APM due to its poor performance.

As shown in Figure 11, all three models exhibit prediction errors exceeding 3%, with the maximum error reaching 5.49%. Such accuracy is insufficient for selecting an optimal layer-skipping strategy for LLM inference. Additionally, these models suffer from severe prediction overhead, including both model construction time and inference time. LR requires 13.5 ms for prediction using 24 inputs, whereas ARIMA takes about 1 second and SVM exceeds 50 seconds.

To address these issues, we find that using the LLM inference execution window as a time unit significantly improves prediction accuracy. Based on this, we use an LR model, which takes the overall rendering headroom from the past three execution windows as input, and predicts the overall rendering headroom for the next execution window. Table II presents the prediction accuracy across all scenarios. The LR model achieves a maximum error of just 1.3% and an average error of only 0.6%. Furthermore, its inference overhead is just 1.3 ms with three input windows, making it well-suited for real-time headroom prediction.

C. Runtime Scheduling

It is important to note that the headroom prediction already accounts for intra-rendering headroom. After predicting the rendering headroom, the scheduler determines an appropriate layer-skipping strategy for the next LLM inference task. It then splits the inference task into smaller subtasks to utilize the fragmented GPU headroom, in a way that adapts sub-task granularity to fit both intra-rendering and inter-rendering headroom. Specifically, Figure 10(b) illustrates the scheduling process employed by the scheduler.

For the intra-rendering headroom, the scheduler employs a feedback-driven scheduling mechanism. It monitors the start and completion of rendering subtasks. When a rendering subtask completes, the scheduler submits a fine-grained LLM inference subtask to utilize the available GPU idle time. Once the inference subtask completes, the scheduler checks whether the next rendering subtask has started computation. If the next rendering subtask has started, the LLM scheduler waits. If the next rendering subtask has not started, the scheduler continues dispatching inference subtasks.

Meanwhile, our analysis shows that the average intra-rendering headroom is 0.24 ms, while 90% of intra-rendering headroom is shorter than 0.73 ms. The total intra-rendering

TABLE III: Experimental specifications.

Software	Windows 11, CUDA driver 566.36 CUDA SDK 12.1, Direct X 12.1, llama.cpp fc83a9e
Hardware	Intel(R) i9-13900KF @ 3.00 GHz, Nvidia RTX 4090
LLM Models	Llama3.2-8B-Instruct, Mistral-7B-Instruct-v0.3
Games	BlackMyth, FFXVI, RDR2

headroom per rendering task averages 1.39 ms, with a maximum of 3.1 ms. Based on this time distribution, we observe that transformer layers could serve as an appropriate subtask granularity during the LLM inference decode phase, as their execution time is approximately 0.4 ms. During the prefill phase, attention layers and FFN layers are better suited, with execution times of 0.5 ms and 1.0 ms, respectively.

To strictly ensure that rendering tasks meet their latency targets, we apply the following runtime scheduling condition: $\sum T_{\text{subtasks}} \leq T_{\text{minimal}}$. Here, T_{subtask} represents the execution time of each dispatched LLM inference subtask, and T_{minimal} represents the minimal inter-rendering headroom of all rendering tasks in the game. By enforcing this constraint at runtime, we guarantee that utilizing intra-rendering headroom does not lead to latency violations for rendering tasks.

Once a rendering task completes, the scheduler switches to coarse-grained LLM subtask scheduling to utilize the inter-rendering headroom. Based on the timestamps of the completed rendering task and the start of the next one, the scheduler can easily determine the size of LLM subtasks, which contain multiple transformer layers.

D. Support for Sudden Spike & Variable-length prompts

We define a sudden spike as an event where the rendering workload between two consecutive frames increases by more than 50%. Experimental results show that only 1.2% of frames exhibit such spikes, and even within these windows, the prediction error remains below 1.3%. This is because, each LLM execution window spans 12-36 frames, making a single-frame spike negligible to the overall headroom prediction. Moreover, multi-frame workload increases can be effectively captured by our temporal prediction model.

To handle severe spikes, we enhance the scheduler to maintain strict QoS. After each token generation, the scheduler updates the temporal prediction with the latest workload data. For instance, after generating the first token, the remaining tokens correspond to a 16-frame execution window, which is used for re-prediction. If a QoS violation risk is detected, the scheduler dynamically adjusts the layer-skipping strategy for subsequent tokens.

To support variable-length prompts, we add a duration predictor for LLM inference task. While the duration prediction is widely studied in previous works [49], [68], [69], we could integrate it into LEGO. Once the inference duration is predicted, we can determine an appropriate layer-skipping strategy for LLM inference based on the available GPU headroom.

VI. IMPLEMENTATION

We utilize llama.cpp [20] as the LLM inference framework. We use Unreal Engine 4 (UE4) [18] as the game engine, with DirectX 12 [34] as the graphics library. We integrate only the front-end of llama.cpp into UE4 and invoke other functions via a dynamic library. Since llama.cpp separates computation graph creation and traversal, we modify the traversal function to incorporate scheduling logic. The engine monitors rendering task state variables and launches inference subtasks upon rendering completion, dispatching transformer layers in the decoding phase, or self-attention and FFN sublayers in the prefilling phase. If no new rendering task arrives, additional inference subtasks continue executing. To support this, we register a new schedulable traversal function in the dynamic library, ensuring correct inference execution.

VII. EVALUATION

A. Experiment Setup

1) *Testbed*: Table III summarizes the software and hardware configurations used in our experiments. Notably, LEGO does not rely on any specialized hardware features of the RTX 4090, making it easily deployable on other gaming GPUs. As shown in Table III, we evaluate LEGO using three popular games and two popular LLM models. Throughout our experiments, all games are configured to run at 60 FPS with high visual settings (4K).

Current consumer-grade gaming GPUs do not support MIG usage. Cloud gaming platforms, like NVIDIA GeForce NOW [42], instead rely on NVIDIA vGPU for time slicing, which only allows static time-slice division. In our experiments, we enhance all baselines with PilotFish, a dynamic time-slice mechanism that enables the LLM inference task to immediately utilize released GPU resources once rendering completes, without waiting for a preassigned slice.

2) Baselines:

- **SmallModel**: A naive solution is to use a smaller model from the same family to balance APM support and accuracy retention. We replace Llama3-8B [16] with Llama3-3B and Mistral-7B [4] with Mistral-4B. At runtime, we partition the LLM inference task into multiple equally sized subtasks based on the average rendering headroom of the game. Once a rendering task completes, the LLM inference task dispatches one subtask for execution.
- **LayerSkip**: We use two layer-skipping methods as baselines: LITE [58] and CALM [52]. Both methods rely on a runtime discrimination mechanism to determine layer skipping based on predefined thresholds. Since LITE achieves better inference accuracy, we use LITE for comparison in all experiments and evaluate both LITE and CALM in Section 7.3. After the layer-skipping strategy is determined, LayerSkip adopts the same scheduling approach as SmallModel.

B. Ensuring FPS and APM

We first demonstrate the effectiveness of LEGO by comparing it with the two baselines across all combinations of game scenarios and LLM models.

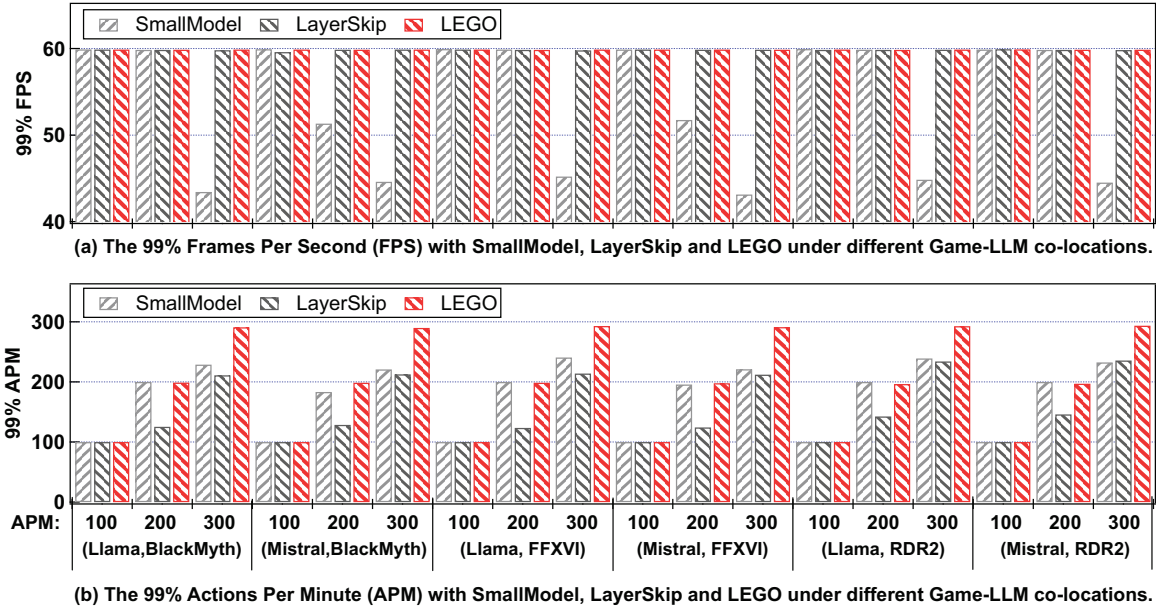


Fig. 12: The 99% FPS and APM with *SmallModel*, *LayerSkip* and *LEGO* under different Game-LLM co-locations.

Figure 12(a) presents the 99th-percentile FPS of the games, while Figure 12(b) shows the 99th-percentile APM of the LLM inference tasks. As shown, *SmallModel* successfully handles the 100 APM and 200 APM scenarios. This is because, in Section 1, we discussed the worst-case scenario using the minimum rendering headroom. However, since rendering tasks can tolerate a slightly delayed start, *SmallModel* is still able to maintain FPS and APM targets in the 100 APM and 200 APM scenarios. However, under the 300 APM scenario, *SmallModel* experiences a 26.2% FPS drop and a 20.5% APM drop, demonstrating its limitations. Additionally, Section 7.3 further highlights the lower inference accuracy resulting from underutilized rendering headroom.

For *LayerSkip*, it successfully handles the 100 APM scenario for the same reason as *SmallModel*. However, in the 200 APM and 300 APM scenarios, it introduces a 28.6% APM drop while maintaining the game’s FPS. This occurs because rendering tasks can tolerate a slightly delayed start. However, since *LayerSkip* does not strictly enforce resource usage constraints, it leads to severe latency violations for LLM inference tasks.

In contrast, *LEGO* successfully maintains both FPS and APM targets across all co-location scenarios. This is because *LEGO* is an algorithm–system co-design specifically tailored for gaming scenarios. On the algorithm side, *LEGO* introduces a resource-oriented layer-skipping adaptor that enables layer-skipping based solely on resource conditions. On the system side, *LEGO* implements a headroom-maximizing scheduler to ensure that the latency targets of both tasks are met at runtime.

C. Inference Accuracy

In this subsection, we present the inference accuracy improvements of *LEGO* compared to baseline methods. Specifically, both *LEGO*’s adaptor and the corresponding modules

TABLE IV: The inference accuracy of *LEGO*, *LITE* and *CALM*.

Method	Dataset	0	4	8	12	13	14	Baseline
LEGO	mmlu	66.8	66.7	66.4	66.3	63.9	40.9	58.2
LEGO	arc-c	76.3	75.0	74.4	73.9	52.2	27.8	73.6
LEGO	squad	70.1	69.2	58.3	57.3	42.0	20.5	39.5
LITE	mmlu	66.8	14.3	11.2	8.7	/	/	58.2
LITE	arc-c	76.3	66.4	60.0	41.0	/	/	73.6
LITE	squad	70.1	41.6	31.9	19.2	/	/	39.5
CALM	mmlu	66.8	/	21.5	/	13.0	/	58.2
CALM	arc-c	76.3	31.9	/	22.4	/	/	73.6
CALM	squad	70.1	26.3	/	2.5	/	/	39.5

in the baseline methods are trained on the same upstream dataset, *WebInstruct*. After training, we evaluate the models on three downstream datasets: *MMLU*, *ARC-C*, and *SQuAD2.0*. *MMLU* and *ARC-C* are evaluated based on accuracy, while *SQuAD2.0* is evaluated using the F1 score.

Table IV presents the inference accuracy of *Llama3-8B* under different layer-skipping settings with *LEGO* support. We do not put the results of *Mistral-7B* due to the page limit, which have a similar effect. In this table, the “skip 0” column represents the original LLM accuracy, while the “baseline” column represents the accuracy of *Llama3-3B*. Notably, *Llama3-8B* achieves the same execution time as *Llama3-3B* when skipping 12 layers.

As shown in the table, *LEGO* consistently outperforms the baseline (*Llama3-3B*) when the number of skipped layers is smaller than 12. This demonstrates that *LEGO* effectively preserves LLM inference accuracy through knowledge distillation. In fact, in the 100 APM and 200 APM scenarios, *LEGO* requires skipping only 5 layers in 90% of cases. In the 300 APM scenario, *LEGO* skips only 13 layers in 80% of cases. Although *LEGO*’s inference accuracy falls below *Llama3-3B* in the 300 APM scenario, *Llama3-3B* experiences severe latency violations under this setting. In contrast, *LEGO*

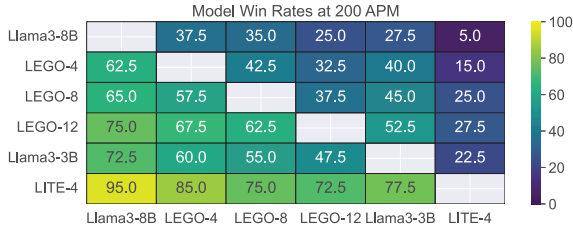


Fig. 13: The win rate heatmap under 200 APM scenario, each cell denotes the win rate of column model over row model.

ensures that both LLM inference and rendering tasks meet their latency targets simultaneously.

In the lower half of Table IV, we compare the inference accuracy of two layer-skipping baselines. Since these methods rely on predefined confidence thresholds for layer skipping, it can be difficult to adjust the thresholds to achieve a desired average number of skipped layers. As shown in the table, both methods suffer significant accuracy degradation due to two key factors. First, these layer-skipping methods use KV replication to fill the KV cache of skipped layers, leading to accuracy loss. Second, these methods discard the knowledge contained in the skipped layers, resulting in direct loss of important information. Based on the table, we calculate the accuracy degradation introduced by various layer-skipping techniques. The results show that LEGO reduces accuracy loss by up to 86.3% when skipping 12 layers, compared to LITE.

D. Effect on Real Gaming

To evaluate the performance of LEGO in a real gaming scenario, we adopt an open-sourced project that evaluates LLMs using *Street Fighter III*. In this project, LLMs control fighters and compete against each other to determine which model performs better in real-time gameplay. Since the game is relatively simple, it does not need the GPU for rendering.

In this experiment, we evaluate the following models: Llama3-8B, Llama3-3B, LEGO-4, LEGO-8, LEGO-12, and LITE-4. Here, LEGO-4 refers to Llama3-8B with 4 layers skipped using LEGO, while LITE-4 refers to Llama3-8B with 4 layers skipped using LITE. Each model pair is evaluated through 40 combat rounds, and all models are configured to operate at 200 APM to ensure a fair comparison of inference accuracy. We do not put the results of 100 APM and 300 APM due to the page limit, which have similar effects.

Figure 13 shows the win rate heatmap among these LLMs. Each cell represents the win rate of the model in the column over the model in the row. For example, the win rate of Llama3-8B over LITE-4 is 95.0%. As expected, Llama3-8B achieves the highest win rate against all other models due to its full parameter and knowledge capacity. Among the reduced models, LEGO-4 consistently outperforms LEGO-8, LEGO-12, Llama3-3B, and LITE-4. Similarly, LEGO-8 surpasses LEGO-12, Llama3-3B, and LITE-4. The win rate of LEGO-12 over Llama3-3B is 47.5%, indicating comparable performance. While LEGO-12 and Llama3-3B exhibit similar inference accuracy, Llama3-3B shows higher win rates against other

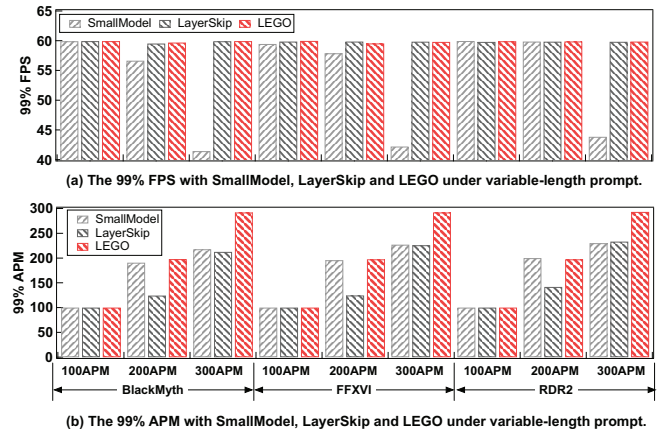


Fig. 14: The 99% FPS and APM with *SmallModel*, *LayerSkip* and LEGO under variable-length prompts.

models compared to LEGO-12. This is likely because LEGO was trained with limited fine-tuning data, whereas Llama3-3B is a mature, well-trained model.

It is important to note that all models in this experiment run at a fixed 200 APM, so the comparison focuses solely on inference accuracy. When applying real layer-skipping traces from *BlackMyth*, *FFXVI*, and *RDR2*, LEGO achieves a 100% win rate over the baselines due to its ability to maintain the target APM under limited resources.

E. Effect vs Nvidia ACE

Nvidia ACE [43] is a LLM-powered AI game companion, which has drawn significant attention. Since the highest priority in gaming is to ensure the player's visual experience, Nvidia ACE proposes a relatively small model, INT4-based Nemotron3-4B. Experimental results show that INT4-based Nemotron3-4B achieves win rates of 5%, 12.5%, 12.5%, 15%, and 15% against Llama3-8B, LEGO-4, LEGO-8, LEGO-12, and Llama3-3B, respectively. All opponents use FP16 precision, as LEGO could utilize more GPU headroom to support FP16 execution. These results demonstrate that LEGO outperforms NVIDIA ACE. Actually, in all experiments, we use an FP16-based Llama3-3B as a stronger baseline.

F. Variable-length Prompts

In this experiment, the input length is uniformly sampled within the range [256, 1024]. Figure 14(a) presents the 99th-percentile FPS of the games, and Figure 14(b) shows the 99th-percentile APM of the LLM inference tasks.

As shown, SmallModel successfully handles the 100 APM scenario but fails under 200 APM and 300 APM. At 200 APM, it experiences a 3.1% FPS drop and a 2.3% APM drop; at 300 APM, the drops increase to 29.3% and 25.0%, respectively. This occurs because SmallModel cannot sustain inference workloads when the input length exceeds 768 under high APM conditions. For LayerSkip, it successfully handles the 100 APM scenario for the same reason as SmallModel. However, under 200 APM and 300 APM, it incurs an average 30.1% APM drop while maintaining game FPS. In contrast,

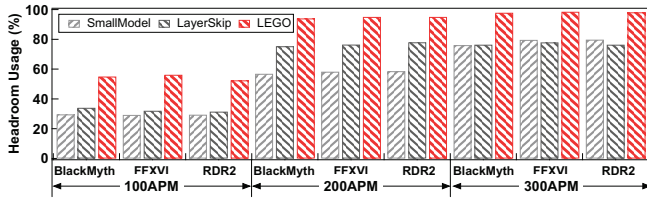


Fig. 15: The headroom usage with *SmallModel*, *LayerSkip* and LEGO under different Game-LLM co-locations.

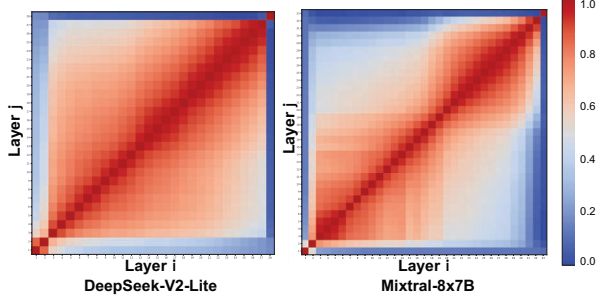


Fig. 16: The similarity heatmaps of two popular MoE Models.

LEGO consistently maintains both APM and FPS across all scenarios, demonstrating its robustness and effectiveness under variable input conditions.

G. Rendering Headroom Usage

Figure 15 presents the headroom usage of *SmallModel*, *LayerSkip*, and LEGO when co-locating Llama models with games under different APM scenarios. As shown in Figure 15, LEGO improves rendering headroom usage by 25.2%, 28.6%, and 18.8% compared to *SmallModel* in the 100 APM, 200 APM, and 300 APM scenarios. Similarly, LEGO achieves 0%, 14.0%, and 16.2% improvement in rendering headroom usage over *LayerSkip* in the three APM scenarios.

In the 100 APM scenario, LEGO shows no improvement over *LayerSkip*. This is because the rendering headroom between rendering tasks alone is sufficient for LLM inference. However, in the 200 APM and 300 APM scenarios, LEGO’s improved headroom usage primarily stems from its ability to utilize GPU idle time within rendering tasks, enabling more efficient execution of LLM inference tasks.

H. MoE Models

We evaluate LEGO on two mainstream MoE models, *DeepSeek-V2-Lite* and *Mixtral-8x7B*. Figure 16 first illustrates the similarity heatmaps of both models. As shown, both MoE models exhibit higher inter-layer similarity in the later layers, consistent with the dense LLMs.

Table V reports the inference accuracy of both models under various layer-skipping configurations and datasets. Since *DeepSeek-V2-Lite* has 28 layers, we test skipping 3, 6, and 9 layers; for *Mixtral-8x7B*, we skip 4, 8, and 12 layers, consistent with the dense LLM settings. As shown, the adaptor effectively preserves inference accuracy when fewer than eight layers are skipped, but accuracy degradation becomes more pronounced at higher skip levels. This occurs because a

TABLE V: The inference accuracy of LEGO under different layer-skipping configurations.

Model	Dataset	Origin	skip-3/4	skip-6/8	skip-9/12
DeepSeek	mmlu	56.6	56.3	56.1	45.1
DeepSeek	arc-c	55.6	55.4	53.2	44.2
DeepSeek	squad	26.8	25.5	23.5	13.1
Mixtral	mmlu	67.8	67.8	67.3	59.9
Mixtral	arc-c	61.7	56.1	50.8	14.9
Mixtral	squad	35.3	32.9	30.4	16.3

TABLE VI: The inference accuracy and Normalized Inference Time of MoE models when adjusting topk.

Model	topk	accuracy	Normalized Inference Time
DeepSeek	6	56.6	100.00%
DeepSeek	4	55.1	93.94%
DeepSeek	2	49.3	88.33%
DeepSeek	1	35.1	84.47%
Mixtral	2	67.8	100.00%
Mixtral	1	61.2	90.71%

substantial portion of knowledge in MoE architectures resides within experts, and removing entire transformer layers disrupts expert routing and representation learning.

We further evaluate *DeepSeek-V2-Lite* and *Mixtral-8x7B* by measuring inference accuracy and latency across different top-k values. As shown in Table VI, reducing top-k maintains reasonable accuracy for MoE models, but execution time does not decrease proportionally. Moreover, since top-k is typically a small integer, it provides only a few discrete adjustment options, corresponding to fixed resource usage levels for LLM inference. This limited flexibility makes quantization poorly suited for dynamic resource conditions in gaming scenarios.

I. Multiple AI Agents

When multiple agents are present (up to nine AI agents in Dota-like games), LLM inference must be executed in batches, which dramatically increases latency. For instance, with an input length of 512, Llama3-8B requires about 400 ms to generate the first token at batch size = 9, while the execution window at 200 APM is only 300 ms. Therefore, when multiple AI agents are required, smaller models such as Llama3-3B or below must be used.

Our method fully supports Llama3-3B with adaptor-based layer skipping. Experimental results show that, at input length = 512, batch size = 9, and output length = 8, LEGO maintains the target FPS and target APM under both 100 APM and 200 APM scenarios. In contrast, *SmallModel* can only support Llama3-1B at 100 APM and fails to meet performance targets at 200 APM. However, LEGO cannot support Llama3-3B at 300 APM, as the LLM inference time (300 ms) exceeds the execution window (200 ms).

J. Justifying APM as a Target

For human players, a high-quality 150 APM gameplay often outperforms a low-quality 200 APM one. However, this observation does not hold for LLMs. Table VII shows the win rate of 150APM Llama3-8B against LEGO-4, LEGO-8, LEGO-12 and Llama3-3B under 200 APM. As shown, 150

TABLE VII: The win rate of Llama3-8B (150 APM) against the opponent models (200 APM).

	LEGO-4	LEGO-8	LEGO-12	Llama3-3B
Llama3-8B (150APM)	7.5%	10%	12.5%	7.5%

APM Llama3-8B achieves a maximum win rate of 12.5%. This is because LLMs with layer skipping could also maintain high inference accuracy whereas the action quality of human players varies widely, often approaching zero. In LLM-powered gaming, APM serves as a user- or developer-defined operational target, assuming that each action is effective. Hence, we adopt APM as the primary performance metric.

K. Overhead and Discussions

Overhead: The overhead of our method primarily arises from two aspects: offline adaptor preparation and online LR model training. For offline adaptor preparation, the overhead depends on the number of adaptors required for a given co-location scenario. BlackMyth, for instance, requires up to 14 LLM adaptors and has a total training time of 36 hours. This offline overhead is negligible.

Each adaptor (an FFN network) occupies 268.8 MB, totaling 3.23 GB for 12 adaptors. The intermediate-result tensor adds 67.5 MB, but since it is required regardless of layer skipping, it incurs no extra memory overhead. At runtime, fitting an LR model with three input windows takes only 0.9 ms, making it suitable for scheduling.

Discussion about Adaptive Rendering Workload: Adaptive rendering techniques are activated only when GPU resources are insufficient. Methods such as Dynamic Resolution Scaling (DRS) and Microsoft Flight Simulator Scaling (MSFS) dynamically reduce rendering workloads to maintain frame rate. In our experiments, all games were configured at maximum graphics settings with DRS enabled, yet no workload adjustment was triggered on the RTX 4090.

Discussion about Quantization and Dynamic Pruning: LEGO is compatible with quantization techniques. Currently, LEGO supports experiments using FP16-based Llama3-8B. After applying INT4 quantization, LEGO can further support LLMs up to 30B parameters. In addition, LEGO can work with any static optimization method, like static quantization and sparsity methods. In contrast, LEGO cannot cooperate with dynamic acceleration methods, since such approaches introduce execution-time uncertainty and additional computational overhead.

Discussion about Small Language Models: Although Small Language Models (SLMs) are becoming increasingly capable, SLMs struggle to handle long-context reasoning and deep compositional tasks. As a result, while SLMs may alleviate some of the challenges that LEGO addresses in simple tasks, LEGO remains essential for more complex scenarios, which require longer contexts, deeper reasoning, and extended computation time under limited resources.

VIII. RELATED WORK

a) Layer Skipping: With large language models, accelerating inference by selectively skipping layers is a key research focus. CALM [52] trains a classifier to assess token consistency and adjusts computational resources dynamically. LITE [58] employs confidence-based early exiting for layer selection. These methods reduce average computation but struggle with dynamic, limited time in Gaming-LLM co-location. In addition, several works [17], [35], [37] have proposed layer-skipping methods to accelerate LLM inference. However, these approaches focus solely on the dynamics of token generation and fail to provide static and stable acceleration suitable for gaming scenarios.

More importantly, LLM-Streamline [13] also proposes to replace consecutive transformer layers with a lightweight network. However, we believe that LLM-Streamline's reasoning process for the design is insufficient. While more than 90% of consecutive transformer layers exhibit over 80% similarity, this alone does not justify skipping them. Our experiments show that skipping discrete layers leads to greater performance degradation than skipping consecutive ones. This is because knowledge is distributed not only within individual transformer layers but also across their inter-layer connections, and skipping discrete layers leads to more knowledge loss.

b) Game Co-location: Several studies have explored co-locating different games [29], [30], [50], [65] and games with other tasks [9], [66]. GAurur [29] employs machine learning to predict performance interference among co-located games. PilotFish [66] integrates cloud gaming with deep learning training, utilizing idle GPU cycles between frames. However, these co-located tasks lack strict runtime requirements, making it impractical to dynamically adjust them based on varying computation times in our scenario.

c) GPU Co-location: Currently, numerous researches focus on improving GPU utilization and optimizing the performance of co-located applications. Several systems have been developed for handling DNN inference [15], [19], [33], [47] and training workloads [31], [62]. TurboTransformers [19] addresses memory allocation and algorithm optimization for variable-length requests. Additionally, some systems are designed to handle both real-time and best-effort tasks in a biased GPU sharing environment [12], [25], [56], [64], [67]. However, these systems generally fail to address the challenges in Game-LLM co-location scenarios where computational resources are dynamic and often insufficient.

IX. CONCLUSION

In this work, we propose LEGO, an algorithm-system co-design that enables the efficient co-location of LLM inference and game rendering tasks. By introducing a resource-oriented layer-skipping adaptor and a headroom-maximizing scheduler, LEGO effectively balances resource utilization while ensuring latency targets for both tasks. Experimental results on an Nvidia RTX 4090 demonstrate significant improvements in rendering headroom usage and LLM inference accuracy.

ACKNOWLEDGMENTS

This work is partially sponsored by the National Key Research and Development Program of China (2024YFB4505703), the National Natural Science Foundation of China (62302302, 62232011), and Natural Science Foundation of Shanghai Municipality (24ZR1430500). This work was supported by Ant Group through CCF-Ant Research Fund. Quan Chen is the corresponding author.

REFERENCES

- [1] "Actions per minute define in lark," https://www.larksuite.com/en_us/topics/gaming-glossary/actions-per-minute-apm, accessed: January 10, 2025.
- [2] "Actions per minute define in wikipedia," https://en.wikipedia.org/wiki/Actions_per_minute, accessed: January 10, 2025.
- [3] "Evaluate llms in real time with street fighter iii," <https://github.com/OpenGenerativeAI/llm-colosseum>, accessed: January 10, 2025.
- [4] "Mistral-7b and mistral-4b," <https://arxiv.org/abs/2310.06825>, accessed: January 10, 2025.
- [5] "Nvidia nsight systems," <https://developer.nvidia.com/nsight-systems>, accessed: January 10, 2025.
- [6] J. A., "How to Run Open-Source LLMs Locally with the OpenAI Connector and Ollama in Mendix," <https://www.mendix.com/blog/how-to-run-open-source-llms-locally-with-the-openai-connector-and-ollama/>, 2024, accessed: 2025-07-27.
- [7] C. Berner, G. Brockman, B. Chan, V. Cheung, P. Debiak, C. Dennison, D. Farhi, Q. Fischer, S. Hashme, C. Hesse *et al.*, "Dota 2 with large scale deep reinforcement learning," *arXiv preprint arXiv:1912.06680*, 2019.
- [8] B. Chen, C. Shu, E. Shareghi, N. Collier, K. Narasimhan, and S. Yao, "Fireact: Toward language agent fine-tuning," *arXiv preprint arXiv:2310.05915*, 2023.
- [9] B. Chen, H. Zhao, W. Cui, Y. He, S. Zhang, Q. Chen, Z. Li, and M. Guo, "Maximizing the utilization of gpus used by cloud gaming through adaptive co-location with combo," in *Proceedings of the 2023 ACM Symposium on Cloud Computing*, 2023, pp. 265–280.
- [10] J. Chen, Y. Lin, S. Peng, S. Wu, K. Kent, H. Dai, K. Ye, and Y. Wang, "Understanding serverless inference in mobile-edge networks: A benchmark approach," *IEEE Transactions on Cloud Computing*, 2024.
- [11] P. Chen, P. Bu, J. Song, Y. Gao, and B. Zheng, "Can vlms play action role-playing games? take black myth wukong as a study case," *arXiv preprint arXiv:2409.12889*, 2024.
- [12] Q. Chen, H. Yang, J. Mars, and L. Tang, "Baymax: Qos awareness and increased utilization for non-preemptive accelerators in warehouse scale computers," *ACM SIGPLAN Notices*, vol. 51, no. 4, pp. 681–696, 2016.
- [13] X. Chen, Y. Hu, J. Zhang, Y. Wang, C. Li, and H. Chen, "Streamlining redundant layers to compress large language models," in *The Thirteenth International Conference on Learning Representations*, 2025.
- [14] P. Cheng, Y. Dai, T. Hu, H. Xu, Z. Zhang, L. Han, N. Du, and X. Li, "Self-playing adversarial language game enhances llm reasoning," *Advances in Neural Information Processing Systems*, vol. 37, pp. 126 515–126 543, 2024.
- [15] A. Dhakal, S. G. Kulkarni, and K. Ramakrishnan, "Gslic: controlled spatial sharing of gpus for a scalable inference platform," in *Proceedings of the 11th ACM Symposium on Cloud Computing*, 2020, pp. 492–506.
- [16] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [17] M. Elhoushi, A. Shrivastava, D. Liskovich, B. Hosmer, B. Wasti, L. Lai, A. Mahmoud, B. Acun, S. Agarwal, A. Roman *et al.*, "Layerskip: Enabling early exit inference and self-speculative decoding," *arXiv preprint arXiv:2404.16710*, 2024.
- [18] Epic Games, "Unreal engine." [Online]. Available: <https://www.unrealengine.com>
- [19] J. Fang, Y. Yu, C. Zhao, and J. Zhou, "Turbotransformers: an efficient gpu serving system for transformer models," in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2021, pp. 389–402.
- [20] ggml, "LlaMa.cpp," <https://github.com/ggml-org/llama.cpp>, 2022, accessed: 2025-07-27.
- [21] Git-disl, "LLM Game Agent Papers," <https://github.com/git-disl/awesome-LLM-game-agent-papers>, 2024, accessed: 2025-07-27.
- [22] Google, "Google recaps how its LLMs could change in-game interactions," <https://the-decoder.com/google-recaps-how-its-llms-could-change-in-game-interactions/>, 2024, accessed: 2025-07-27.
- [23] J. Gregory, *Game engine architecture*. AK Peters/CRC Press, 2018.
- [24] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, and Y. Zhu, "Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.
- [25] M. Han, H. Zhang, R. Chen, and H. Chen, "Microsecond-scale preemption for concurrent {GPU-accelerated}{DNN} inferences," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 539–558.
- [26] InZOI, "Creating Next-Gen Agents in KRAFTON's inZOI," https://schedule.gdconf.com/session/creating-next-gen-agents-in-kraftons-inzoi-presented-by-nvidia/911189?utm_source=chatgpt.com, 2025, accessed: 2025-10-21.
- [27] I. Lambe, "The NEW Surprising Number of Steam Games that Use GenAI," <https://www.totallyhuman.io/blog/the-surprising-new-number-of-genai-games-on-steam>, 2025, accessed: 2025-10-21.
- [28] J. Leandro, S. Rao, M. Xu, W. Xu, N. Jovic, C. Brockett, and B. Dolan, "Geneva: Generating and visualizing branching narratives using llms," in *2024 IEEE Conference on Games (CoG)*. IEEE, 2024, pp. 1–5.
- [29] Y. Li, C. Shan, R. Chen, X. Tang, W. Cai, S. Tang, X. Liu, G. Wang, X. Gong, and Y. Zhang, "Gaugur: Quantifying performance interference of colocated games for improving resource utilization in cloud gaming," in *Proceedings of the 28th international symposium on high-performance parallel and distributed computing*, 2019, pp. 231–242.
- [30] Y. Li, C. Zhao, X. Tang, W. Cai, X. Liu, G. Wang, and X. Gong, "Towards minimizing resource usage with qos guarantee in cloud gaming," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 2, pp. 426–440, 2020.
- [31] G. Lim, J. Ahn, W. Xiao, Y. Kwon, and M. Jeon, "Zico: Efficient {GPU} memory sharing for concurrent {DNN} training," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, 2021, pp. 161–175.
- [32] C. Lin, Z. Han, C. Zhang, Y. Yang, F. Yang, C. Chen, and L. Qiu, "Parrot: Efficient serving of {LLM-based} applications with semantic variable," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 929–945.
- [33] Z. Liu, J. Leng, Z. Zhang, Q. Chen, C. Li, and M. Guo, "Veltair: towards high-performance multi-tenant deep learning services via adaptive compilation and scheduling," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 388–401.
- [34] F. Luna, *Introduction to 3D game programming with DirectX 12*. Mercury Learning and Information, 2016.
- [35] X. Luo, W. Wang, and X. Yan, "Adaptive layer-skipping in pre-trained llms," *arXiv preprint arXiv:2503.23798*, 2025.
- [36] W. Ma, Q. Mi, Y. Zeng, X. Yan, Y. Wu, R. Lin, H. Zhang, and J. Wang, "Large language models play starcraft ii: Benchmarks and a chain of summarization approach," *arXiv preprint arXiv:2312.11865*, 2023.
- [37] X. Men, M. Xu, Q. Zhang, B. Wang, H. Lin, Y. Lu, X. Han, and W. Chen, "Shortgpt: Layers in large language models are more redundant than you expect," *arXiv preprint arXiv:2403.03853*, 2024.
- [38] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, "Locating and editing factual associations in GPT," *Advances in Neural Information Processing Systems*, vol. 35, pp. 17 359–17 372, 2022.
- [39] NetEase, "NetEase to add game version of ChatGPT to Justice Online Mobile for dialogue generation and unique reactions," https://gameworldobserver.com/2023/02/16/netease-chatgpt-justice-online-mobile-dialogue-generation/?utm_source=chatgpt.com, 2025, accessed: 2025-10-21.
- [40] A. News, "Can AI make video games more immersive?" <https://apnews.com/article/ai-artificial-intelligence-video-games-npc-c1327bb9130136d0a5f658f44176c5e7>, 2024, accessed: 2025-07-27.
- [41] Nvidia, "RTX 4090 whitepaper," <https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf>, 2023, accessed: 2025-07-27.
- [42] Nvidia, "GeForce NOW," <https://www.nvidia.com/en-sg/geforce-now/>, 2025, accessed: 2025-10-21.
- [43] Nvidia, "NVIDIA ACE for Games," <https://developer.nvidia.com/ace-for-games>, 2025, accessed: 2025-10-21.

- [44] Nvidia, "NVIDIA Announces First Digital Human Technologies On-Device Small Language Model," <https://blogs.nvidia.com/blog/digital-human-technology-mecha-break/>, 2025, accessed: 2025-10-21.
- [45] Nvidia, "NVIDIA Redefines Game AI With ACE Autonomous Game Characters," <https://www.nvidia.com/en-us/geforce/news/nvidia-ace-autonomous-ai-companions-pubg-naraka-bladepoint/>, 2025, accessed: 2025-10-21.
- [46] Nvidia, "RTX 5090 whitepaper," <https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>, 2025, accessed: 2025-07-27.
- [47] C. Olston, N. Fiedel, K. Gorovoy, J. Harmsen, L. Lao, F. Li, V. Rajashekhar, S. Ramesh, and J. Soyke, "Tensorflow-serving: Flexible, high-performance ml serving," *arXiv preprint arXiv:1712.06139*, 2017.
- [48] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior. arxiv," *arXiv preprint ArXiv:2304.03442*, 2023.
- [49] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative llm inference using phase splitting," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 118–132.
- [50] Z. Qi, J. Yao, C. Zhang, M. Yu, Z. Yang, and H. Guan, "Vgris: Virtualized gpu resource isolation and scheduling in cloud gaming," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 11, no. 2, pp. 1–25, 2014.
- [51] S. Sarfaraz, "1 in 5 Steam games released in 2025 use Generative AI," <https://tech4gamers.com/1-in-5-steam-games-in-2025-use-generative-ai/>, 2025, accessed: 2025-10-21.
- [52] T. Schuster, A. Fisch, J. Gupta, M. Dehghani, D. Bahri, V. Tran, Y. Tay, and D. Metzler, "Confident adaptive language modeling," *Advances in Neural Information Processing Systems*, vol. 35, pp. 17456–17472, 2022.
- [53] X. Shao, W. Jiang, F. Zuo, and M. Liu, "Swarmbrain: Embodied agent for real-time strategy game starcraft ii via large language models," *arXiv preprint arXiv:2401.17749*, 2024.
- [54] C. Siebler, "Optimizing latency for Azure OpenAI Service," <https://clemenssiebler.com/posts/optimizing-latency-azure-openai/>, 2023, accessed: 2025-07-27.
- [55] Steam, "AI_Roguelite," https://store.steampowered.com/app/1889620/AI_Roguelite/, 2025, accessed: 2025-10-21.
- [56] F. Strati, X. Ma, and A. Klimovic, "Orion: Interference-aware, fine-grained gpu sharing for ml applications," in *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024, pp. 1075–1092.
- [57] P. Sweetser, "Large language models and video games: A preliminary scoping review," in *Proceedings of the 6th ACM Conference on Conversational User Interfaces*, 2024, pp. 1–8.
- [58] N. Varshney, A. Chatterjee, M. Parmar, and C. Baral, "Accelerating llama inference by enabling intermediate layer decoding via instruction tuning with lite," *arXiv preprint arXiv:2310.18581*, 2023.
- [59] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in neural information processing systems*, 2017.
- [60] T. Verge, "Nvidia's AI NPCs are no longer chatbots — they're your new PUBG teammate," https://www.theverge.com/2025/1/6/24337949/nvidia-ace-ai-npcs-pubg-ally-teammate?utm_source=chatgpt.com, 2025, accessed: 2025-10-21.
- [61] C. Vohera, H. Chheda, D. Chouhan, A. Desai, and V. Jain, "Game engine architecture and comparative study of different game engines," in *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. IEEE, 2021, pp. 1–6.
- [62] G. Wang, K. Wang, K. Jiang, X. Li, and I. Stoica, "Wavelet: Efficient dnn training with tick-tock scheduling," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 696–710, 2021.
- [63] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.
- [64] W. Xiao, S. Ren, Y. Li, Y. Zhang, P. Hou, Z. Li, Y. Feng, W. Lin, and Y. Jia, "{AntMan}: Dynamic scaling on {GPU} clusters for deep learning," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 533–548.
- [65] C. Zhang, J. Yao, Z. Qi, M. Yu, and H. Guan, "vgasa: Adaptive scheduling algorithm of virtualized gpu resource in cloud gaming," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 11, pp. 3036–3045, 2013.
- [66] W. Zhang, B. Chen, Z. Han, Q. Chen, P. Cheng, F. Yang, R. Shu, Y. Yang, and M. Guo, "{PilotFish}: Harvesting free cycles of cloud gaming with deep learning training," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, 2022, pp. 217–232.
- [67] W. Zhang, W. Cui, K. Fu, Q. Chen, D. E. Mawhirter, B. Wu, C. Li, and M. Guo, "Laius: Towards latency awareness and improved utilization of spatial multitasking accelerators in datacenters," in *Proceedings of the ACM international conference on supercomputing*, 2019, pp. 58–68.
- [68] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "{DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 193–210.
- [69] K. Zhu, Y. Gao, Y. Zhao, L. Zhao, G. Zuo, Y. Gu, D. Xie, Z. Ye, K. Kamahori, C.-Y. Lin *et al.*, "{NanoFlow}: Towards optimal large language model serving throughput," in *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, 2025, pp. 749–765.